

Progressive Disclosure

Learning objectives:

- Use progressive disclosure in AGENTS.md
- Describe why we keep AGENTS.md short
- Use emojis to track which knowledge documents are used

[emily bache](#)

emilybache.com

[@emilybache-tech-coach](#)

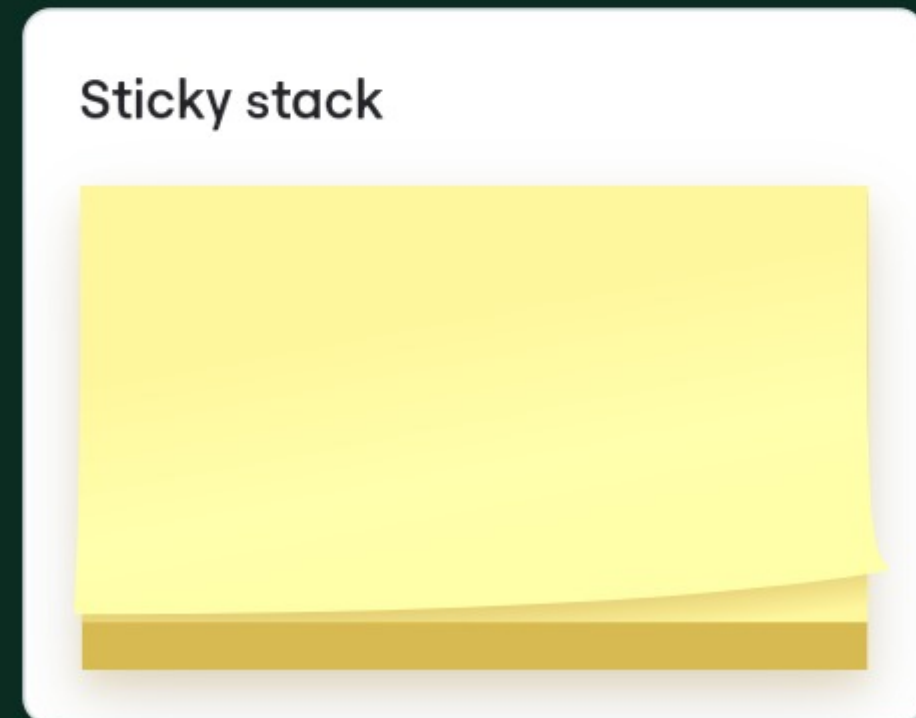
(she/her)

Happy coding!



What kinds of information do you put in AGENTS.md?

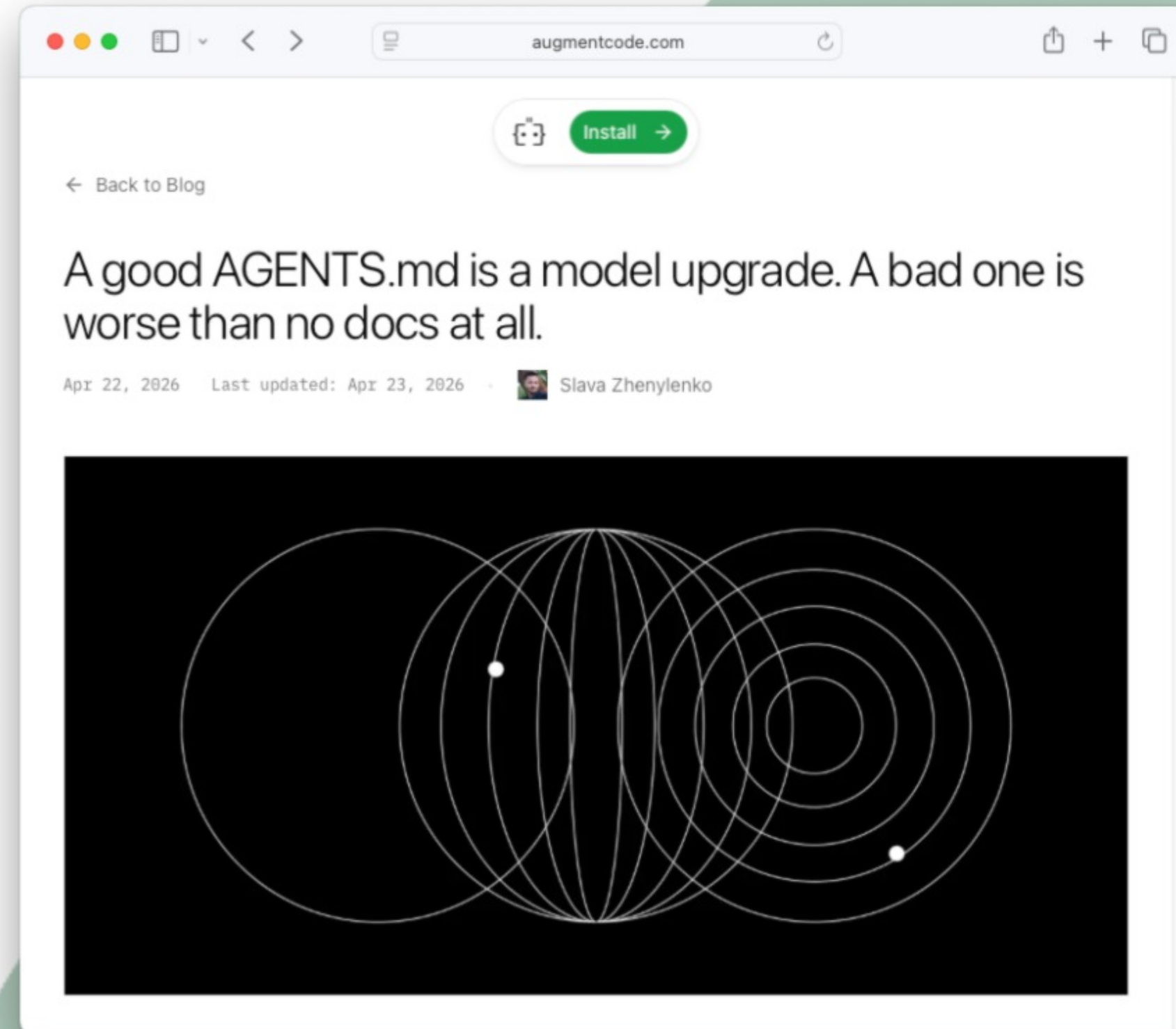
- Give me three examples



Advice for AGENTS.md

"a well-written AGENTS.md is worth roughly a model upgrade, with the best files lifting performance 10-15% over teams running no file at all." ([source](#))

- 100 - 150 lines is optimal



The Agent Loop

1: Prompt and
reference files

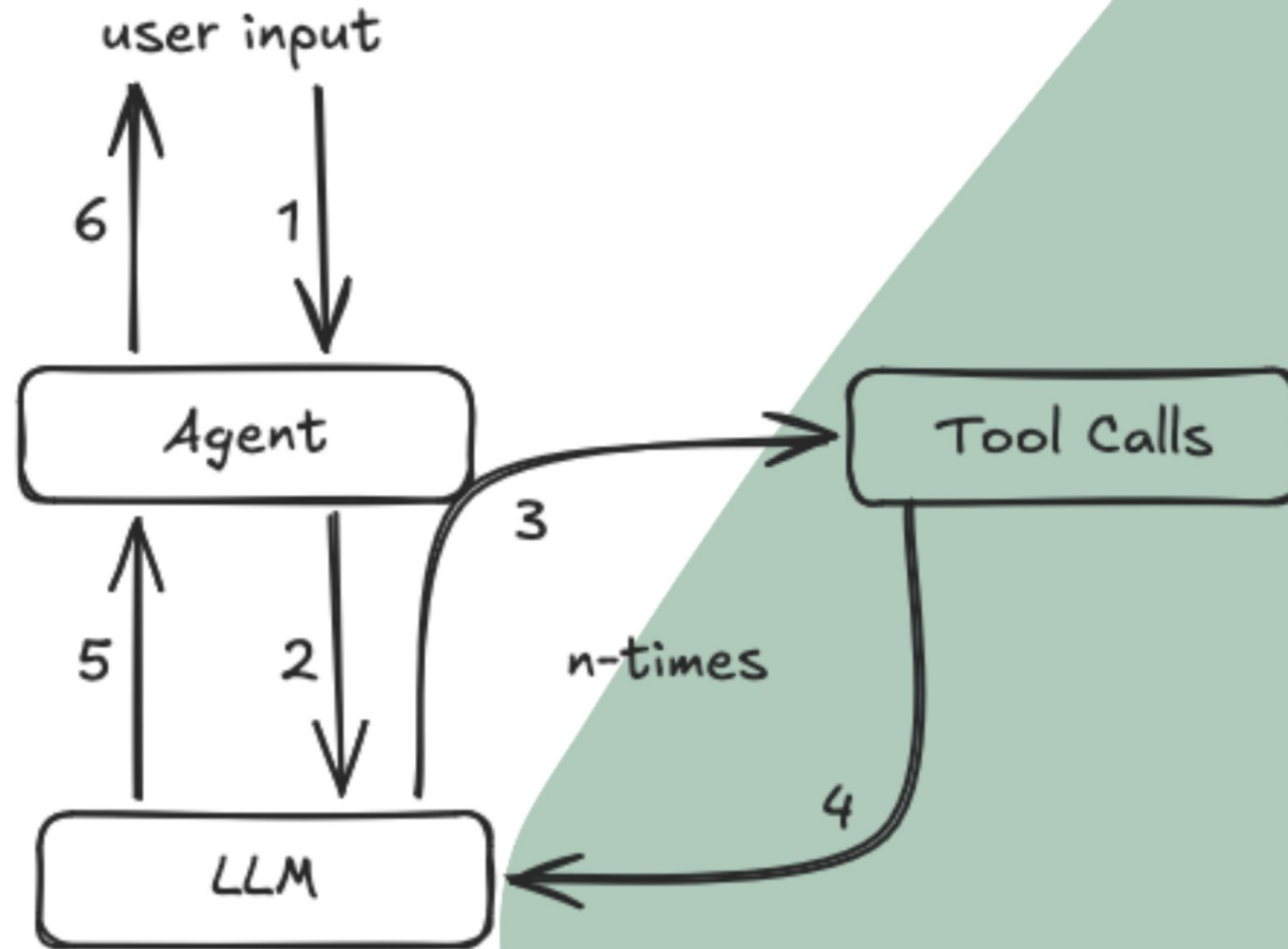
2: Instructions
to LLM

3: Tool
instructions

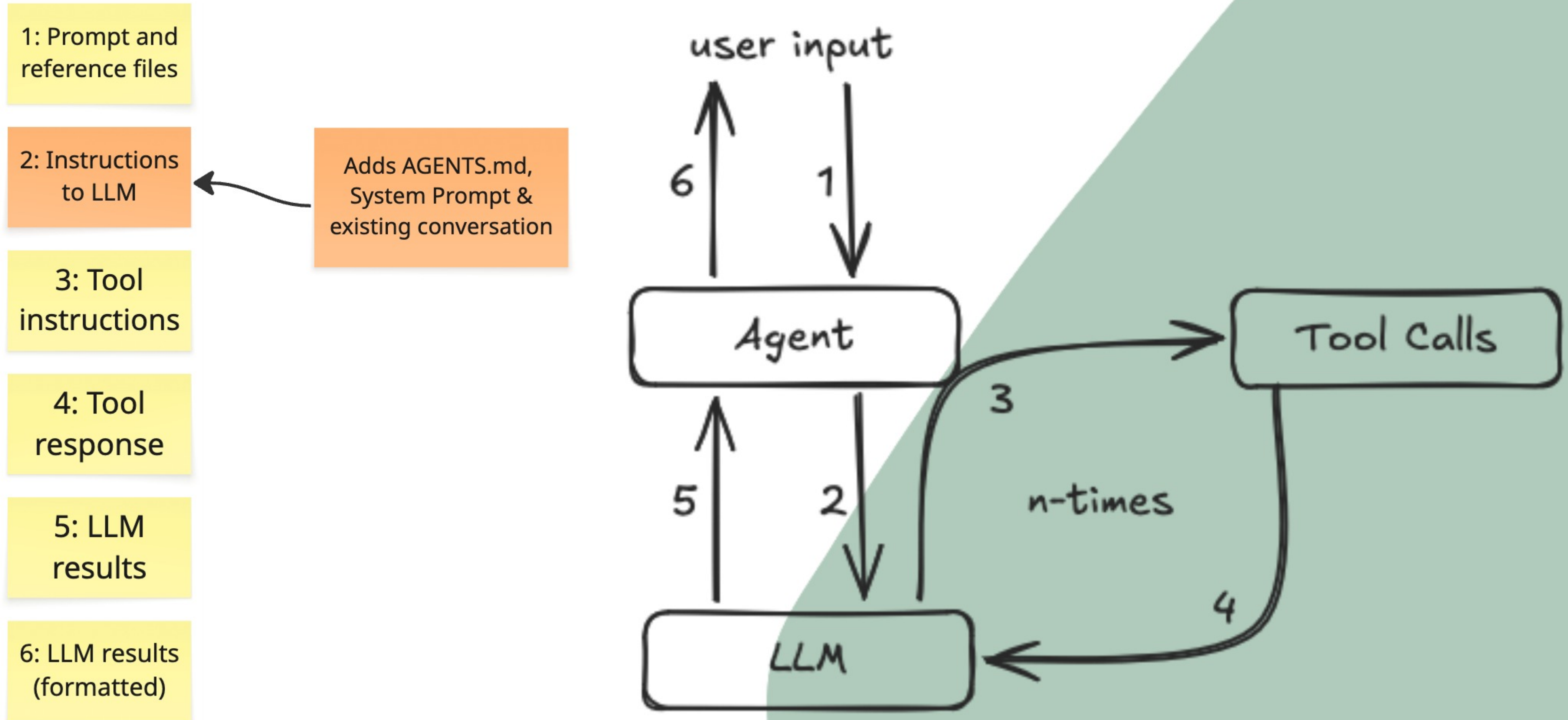
4: Tool
response

5: LLM
response

6: LLM response
(formatted)

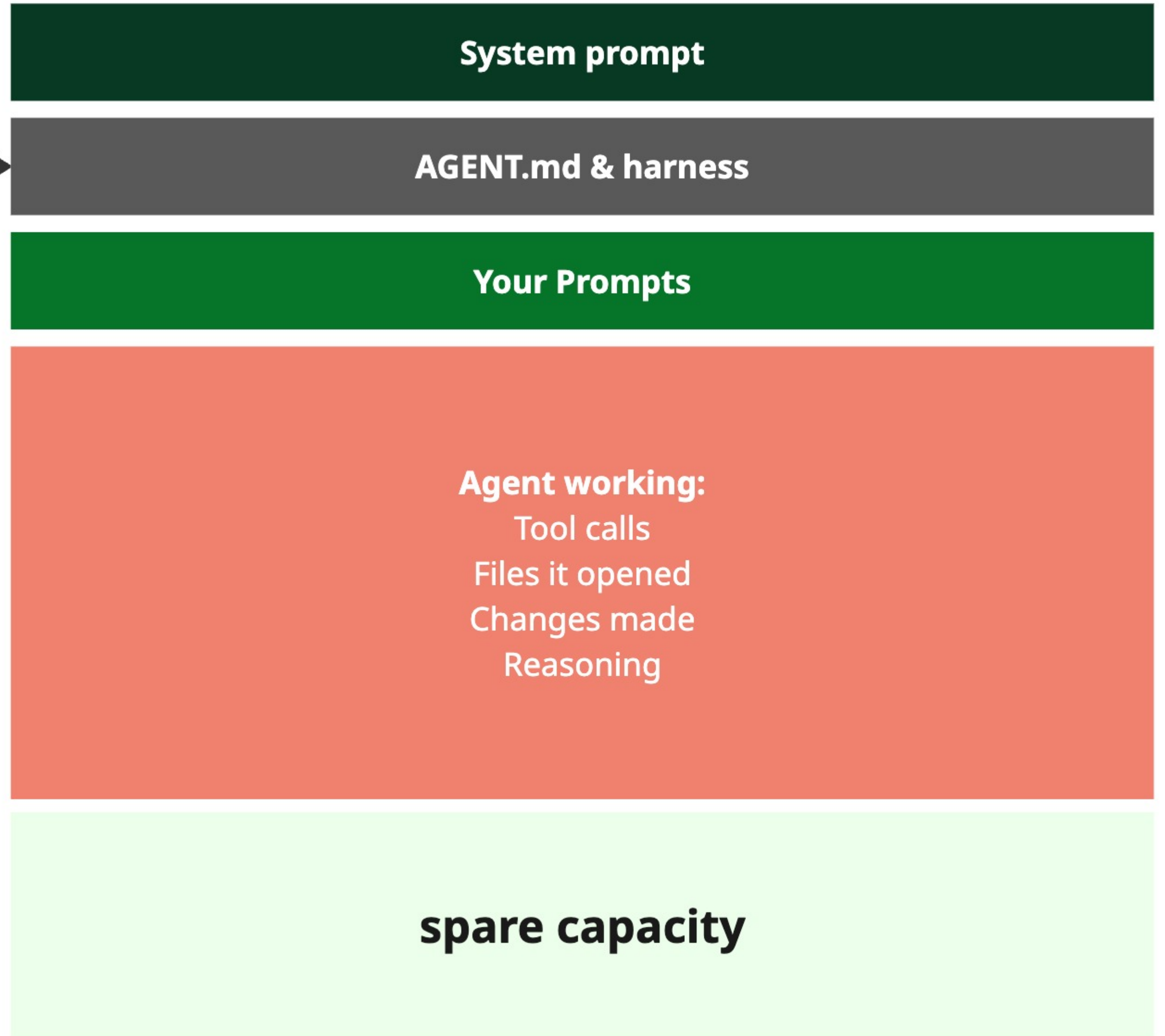


The Agent Loop



Instructions to the LLM: Context Window

Can take a
lot of space



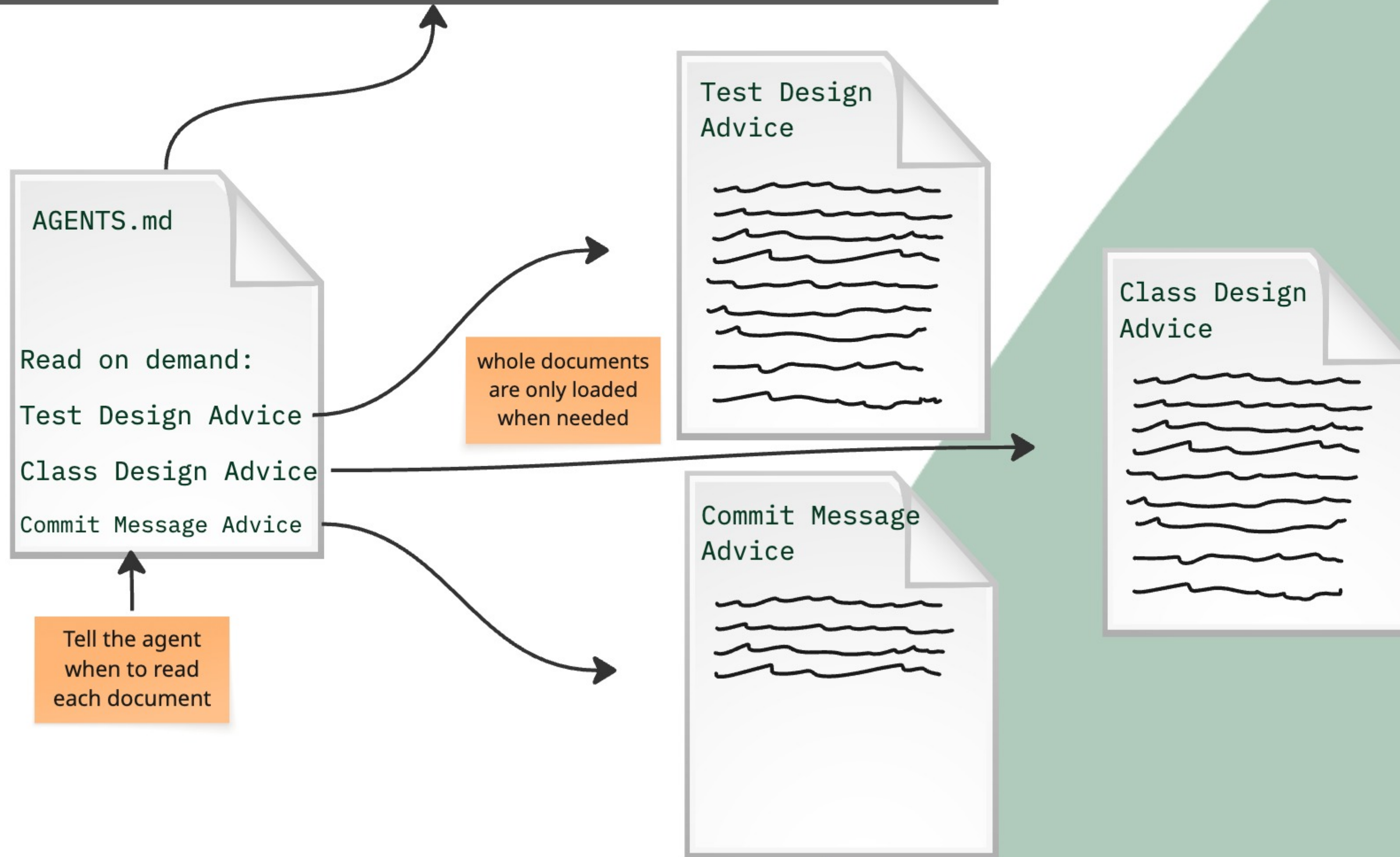
Demo - *Progressive Disclosure*

Improve the
design of this
code



Progressive Disclosure: Minimize Context

AGENT.md & harness



Coding exercise - *Progressive Disclosure*

Practice using different knowledge documents



Instructions

- Use this short, simple prompt and refer to one file of sourcecode.
- Inspect the agent output.
 - Which knowledge documents did it use?
 - Did it do as you wished? If not, improve the document(s) and try again.
- When you have code that is good enough to commit, create a new knowledge document for writing commit messages and use it.

Improve the
design of this
code

Work on: <https://github.com/emilybache/Encode-TestDesign-Kata>

Agent instructions

- Note down useful commands, instructions, prompts, or IDE shortcuts to share with the team!

Sticky stack



Copilot
commands
/context
/instructions

Claude
commands
/context all
/memory

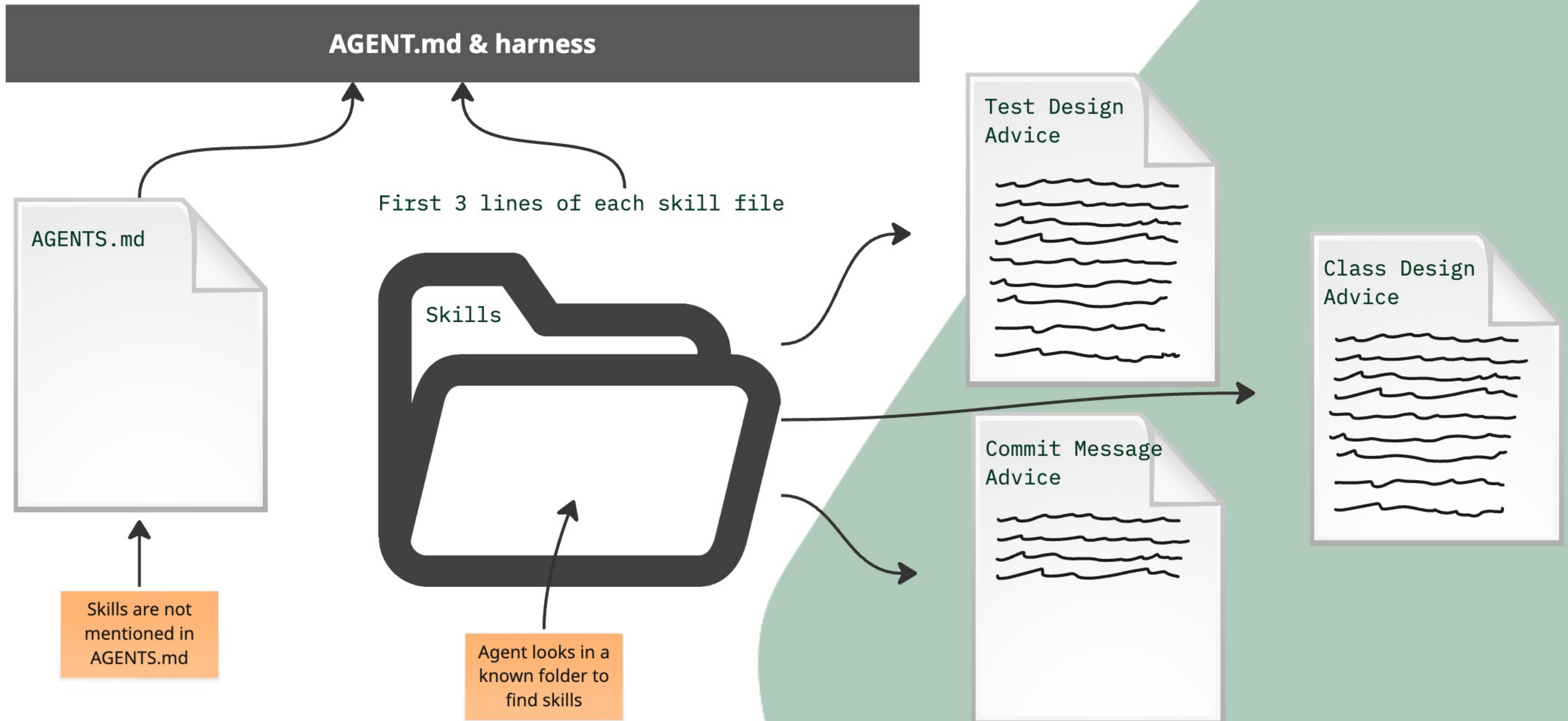
show emojis
in intellij idea:
CTRL+
COMMAND+
SPACE

test-design instructions:
- use camel case for test names.
- Each test has a clear before/after or input/output.
- Each test has a single motivation or associated business rule.
- Test names should read as plain English.
- Use the language of the problem domain
- assert only one thing per test
- Each test is responsible for constructing its own context. Do not rely on class-level setup (@BeforeEach, @BeforeAll) for state that varies between tests.
- Avoid @AfterEach/@AfterAll teardown. Needing cleanup is a signal that the test is not properly isolated — prefer designs where tests leave no state behind.
- Use private helper methods to keep construction readable without sharing mutable state across tests.
https://github.com/etudevelopers/etude_platform/pull/5215

production code design instructions:
- use camel case for test names.
- Each test has a clear before/after or input/output.
- Each test has a motivation or associated business rule or rules,
- Use the language of the problem domain
- assert only one thing per test

commit-convention instructions:
- Always present the commit message to the user for confirmation before committing.
- Write commit messages as: a short imperative subject line (max 50 chars), a blank line, then a body explaining what changed and why.

Progressive Disclosure: "Skills"



Conclusions

- What is Progressive Disclosure and why is it useful for coding agents?



useful to keep the context window for agents in a manageable size - prevent context bloat

Conditionally loading only relevant information to get more relevant outcome without blowing up context window

keep emojis at bay 🤪

deal with context issues

A context window only uses the references necessary for the work at hand

progressive emojis disclosure

Manage attention

A bloated context window reduces performance and increases cost unnecessarily

to keep context window small

to structure knowledge documents, avoid having all in one big file

keep context small, kind of "lazy loading" necessary instructions

allows placing files in different folders, for example for test or